



DSA0405 - FUNDAMENTALS OF DATA SCIENCE

Data Manipulation with Pandas

Your Toolkit for Insight — master the essential library that powers data science workflows in Python.

PYTHON · DATA SCIENCE · PANDAS

Presented by:
BHARATH M
192324302



Why Pandas? Understanding Its Power



Structured Data

Work with tabular data effortlessly — rows, columns, and indices feel native.



Speed & Flexibility

Built on NumPy, Pandas handles millions of rows with optimized performance.



Ecosystem Ready

Integrates seamlessly with NumPy, Matplotlib, Scikit-learn, and more.



Industry Standard

The go-to library for data wrangling in finance, research, and tech.

Getting Started: Your First DataFrame



Install & Import

```
pip install pandas  
import pandas as pd
```

Create a DataFrame

```
df = pd.DataFrame({  
    "name": ["Alice", "Bob"],  
    "score": [95, 87]  
})
```

📌 A **DataFrame** is Pandas' core structure — like a spreadsheet with superpowers.

Selecting & Filtering Data

Pinpoint exactly the rows and columns you need — no more scrolling through thousands of records.

1

Column Selection

```
df["name"]  
df[["name", "score"]]
```

2

Row Filtering

```
df[df["score"] > 90]  
df.query("score >= 85")
```

3

Label-Based Access

```
df.loc[0, "name"]  
df.iloc[0:3, 1]
```

4

Conditional Masks

```
mask = df["score"].isna()  
df[~mask]
```

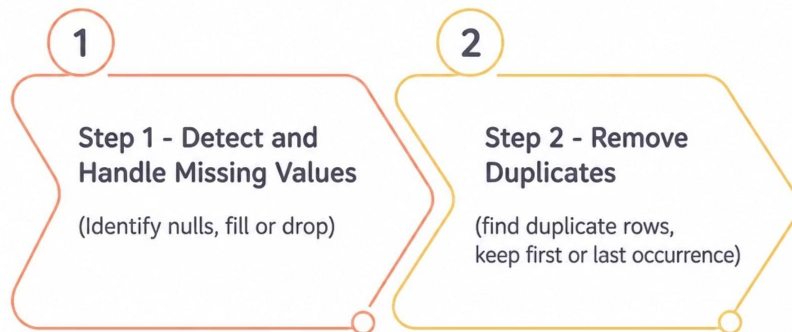

Cleaning Data: Missing Values & Duplicates

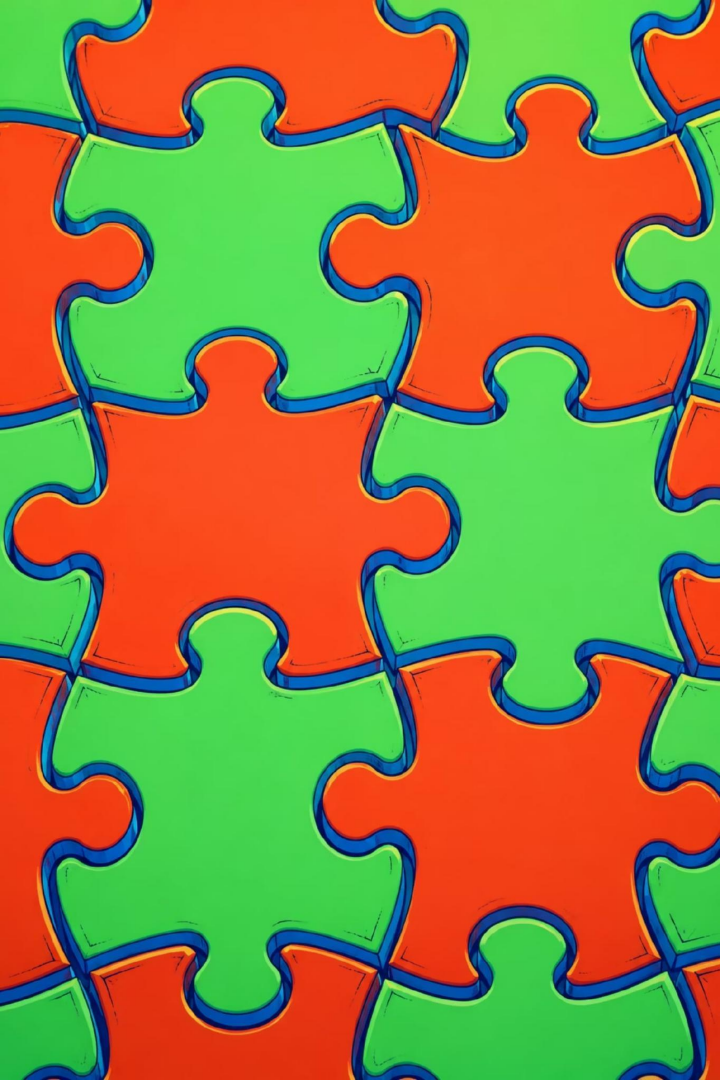
Handle Missing Values

- **Detect:** `df.isna().sum()`
- **Fill:** `df.fillna(0)`
- **Drop:** `df.dropna()`
- **Forward-fill:** `df.ffill()`

Remove Duplicates

```
df.duplicated().sum()  
df.drop_duplicates()
```





Transforming Data: Reshape, Merge & Group

1

Reshape

`melt()` and `pivot()` convert between wide and long formats.

2

Merge

`merge()` joins DataFrames like SQL — inner, left, right, outer.

3

Group

`groupby()` aggregates data by category for summary statistics.

Advanced Operations: Apply & Pivot

Apply Custom Functions

```
df["grade"] = df["score"].apply(  
    lambda x: "A" if x >= 90 else "B"  
)
```

Use `apply()` row-wise or column-wise for custom transformations.

Pivot Tables

```
df.pivot_table(  
    values="score",  
    index="name",  
    aggfunc="mean"  
)
```

map()

Replace values using a dictionary or Series.

replace()

Swap specific values across the DataFrame.

transform()

Apply group-level operations while preserving shape.

Time Series Analysis with Pandas

Temporal data is everywhere — sales, sensor readings, stock prices. Pandas makes it manageable.



Parse Dates

`pd.to_datetime()` converts strings to datetime objects for time-based indexing.



Resample & Roll

`df.resample("M").mean()` aggregates by time periods; `rolling()` computes moving averages.

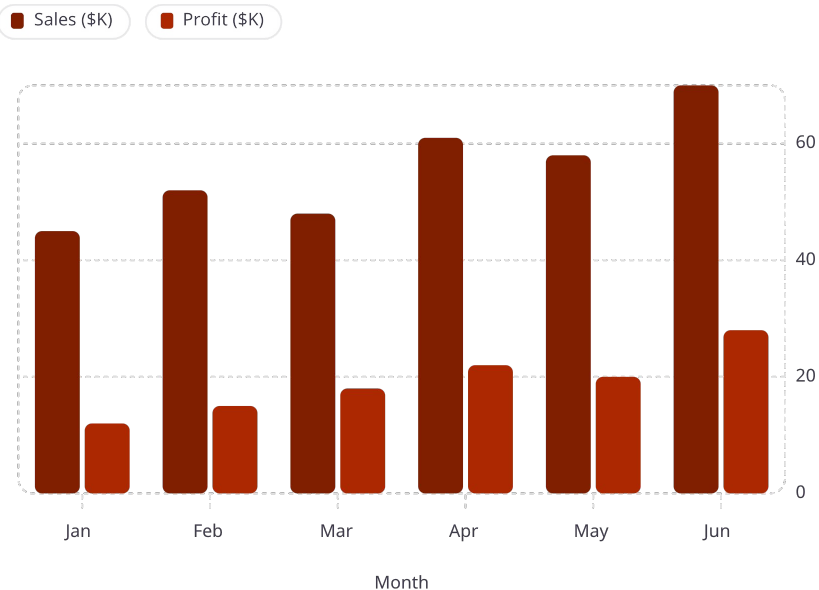


Shift & Diff

`shift()` and `diff()` reveal trends, growth rates, and period-over-period changes.



Visualization with Pandas



Built-In Plotting

Pandas DataFrames plot directly using Matplotlib under the hood.

- **Line:** `df.plot()`
- **Bar:** `df.plot.bar()`
- **Histogram:** `df.hist()`
- **Scatter:** `df.plot.scatter()`



For advanced visuals, pair Pandas with **Seaborn** or **Plotly**.



Best Practices & Next Steps



Chain Thoughtfully

Use method chaining for readability, but avoid excessive `inplace=True`.



Validate Early

Run `df.info()` and `df.describe()` immediately after loading data.



Keep Learning

Explore the **official Pandas docs**, **Kaggle courses**, and real-world datasets.



Level Up

Next: explore **Polars** for speed, **Dask** for scale, and **PyArrow** for efficiency.